

RAG和Agent相关面试题

第二次面试

- 1、对自己的项目和研究方向，还有算法的方向做一个自我介绍？
- 2、RAG项目在数据分块这里采用了什么策略？
- 3、RAG系统中，数据分块后，进行Overlap设置为多长？
- 4、在RAG的检索阶段，有哪些方法提高检索阶段的召回率呢？有尝试过比如说去优化它的相似度计算这些吗？
- 5、你了解RAG检索阶段这些常用的算法，如ANN算法，KD树这些吗？
- 6、简单说下什么是KVCache，什么是VLLM框架？
- 7、大模型推理时，一般分为哪几个阶段，比如Prefill 和 Decode两个阶段，就是预填充和解码
- 8、VLLM框架为什么能提高吞吐量？
- 9、一般做Agent时，一般会怎样构建一个多Agents系统，会考虑哪些方面？
- 10、你这个主Agent，就是这个全科分诊Agent，是怎么保证主Agent把任务拆解的比较合理呢？怎样提高主Agent的分流、准确性。怎样用主Agent对用户的输入对分科Agent做一个匹配？就是让它的分流比较准确？
- 11、BERT在预训练时用到了哪些任务呢？每个任务都用了怎么样的损失函数？
- 12、微调的方法有哪些？LoRA微调和P-Tuning分别适用于哪些场景？
- 13、RLHF的PPO算法的训练流程是什么？
- 14、常用的大语言模型有哪些？他们每一个架构做了哪些改进有了解吗？
- 15、多模态有了解吗？
- 16、算法基础和数据结构这块怎么样？
- 17、一个数组中，找长度为3的递增子序列，是则返回True，不是则返回False，复杂度是多少？

1 RAG项目中的数据分块策略

一、RAG 为何要做“数据分块”？

因为：

LLM 的输入长度有限（context window），整篇文档无法直接入检索库，需要切分成较短的、语义完整的 **Chunk 单元**，才能用于向量化、召回与回答。

二、常见的数据分块策略（实战常用 5 种）

策略类型	说明	优缺点	📄
1. 固定长度分块（Fixed size）	每 N 个字符、token 分一块（如 512 tokens）	简单、效率高，但可能语义断裂	
2. 滑动窗口（Sliding Window）	每块有重叠，如：块1=[1-512]，块2=[256-768]	提高上下文完整性，但冗余数据多	
3. 按自然段分（Paragraph-based）	遇换行/标点切	语义完整好，但每段长度不一	
4. 按标题/小节划分（Semantic Split）	医疗类如“适应症”、“用法用量”作为分段依据	结构化文档效果好，但依赖解析规则	
5. 动态切块（Token-aware chunking）	使用分词器，按 token + 语义断点切块（结合句子末尾等）	保语义 + 控 token 数，实际效果最好但复杂	

1.1 固定长度分块

一、什么是固定长度分块（Fixed-length Chunking）

固定长度分块是一种最常用、最简单的文本切分策略：

把每篇原始文档直接按**字符数**或**token数**等长度单位，等长切分成多个段落。

例如：

- 每 512 字符/Token 为一个 Chunk；
- 设置 chunk_overlap（重叠区域）以增强语义连续性。

二、优缺点分析

优点	缺点
✓ 实现简单，效率高，适用于大规模数据处理	✗ 不考虑语义边界，可能把一句话切成两半
✓ 易于向量化、索引统一处理	✗ 上下文可能断裂，影响语义检索/生成
✓ 易于并行切分/批量处理	✗ 可能存在大量无效chunk（如单独标点段）

三、参数设置建议

参数	建议设置	说明
chunk_size	256 ~ 1024 tokens 256~1024令牌	LLM 一般支持上下文 4K ~ 16K，可选 512 较通用
chunk_overlap	50~200 tokens 50~200令牌	保留一定重叠提升上下文连续性
unit	token or character 令牌或性格	token 精度更高，但需 tokenizer 支持

四、完整代码示例

方法一：纯 Python 实现（按字符长度分块）

```
def split_text_fixed(text, chunk_size=512, chunk_overlap=128):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        chunks.append(chunk)
        start += chunk_size - chunk_overlap # 滑动窗口
    return chunks

# 示例
text = "这是一个非常长的医疗文档内容....." * 50
chunks = split_text_fixed(text, chunk_size=512, chunk_overlap=128)

for i, chunk in enumerate(chunks[:3]):
```

```
print(f"Chunk {i+1}:\n{chunk[:100]}...\n")
```

✅ 方法二：使用 LangChain 中的 CharacterTextSplitter 实现

LangChain 默认支持按字符长度切分的 `CharacterTextSplitter` 类，非常适合固定长度分块。

```
from langchain.text_splitter import CharacterTextSplitter
from langchain.schema.document import Document

text_splitter = CharacterTextSplitter(
    separator="", # 不按标点切，完全固定长度
    chunk_size=512,
    chunk_overlap=128
)

raw_text = "这是一个典型的医学说明书文本，包含用药剂量、适应症、注意事项等内容....." * 20
docs = text_splitter.split_documents([Document(page_content=raw_text)])

# 展示前几个chunk
for i, doc in enumerate(docs[:3]):
    print(f"Chunk {i+1}:")
    print(doc.page_content[:100], "...\\n")
```

✅ 五、工程使用建议

✅ 分块后的处理流程（典型RAG）：

1. 每个 chunk 用 BGE 等模型做向量化
2. 存入 FAISS / Milvus / Weaviate 向量库
3. 检索时，按 query 向量召回 Top-K chunk
4. 拼接 chunk 内容，传入 LLM 生成答案

✅ 六、面试推荐答法（模板）

“在我们构建的 RAG 医疗问答系统中，向量召回部分需要对文档进行分块处理。我们采用了固定长度分块的方式，每个 chunk 为 512 字符，设置了 128 字符重叠。这样可以快速、高效地构建 embedding 向量库。虽然这种方式不能完全保证语义连续性，但通过增加 `chunk_overlap` 可以缓解这个问题。我们在实验中也测试过 sentence-based 分块，但在大规模处理时固定长度方案更易于部署与索引构建。”

1.2 滑动窗口分块

一、什么是滑动窗口分块？

滑动窗口分块是一种通过设置固定长度 `chunk_size` 和一定长度的重叠 `chunk_overlap`，逐步滑动窗口进行文本切片的方法。

例子：

- `chunk_size = 512`
- `chunk_overlap = 128`

那么切分过程如下：

Chunk序号	起始位置	内容范围
Chunk 1 块1	0	[0, 512)
Chunk 2 块2	384	[384, 896)
Chunk 3 块3	768	[768, 1280)

也就是每一段有 128 token 是与上一段重复的。这能保证回答模型上下文不丢失，尤其对于长句子、跨段落内容非常有帮助。

二、滑动窗口的优点和缺点

优点	缺点	
✓ 保持上下文连续性，减少“语义断句”风险	✗ 有重复数据（向量存储量稍增）	📄
✓ 提高召回的命中率（每个片段更完整）	✗ 有一定的重复计算成本	
✓ 与 BERT、BGE 等模型兼容性好（Token-aware）	✗ 文档较大时索引更多 chunk，需考虑性能优化	

六、面试题模版（建议背诵）

“在我们医疗 RAG 问答系统中，我们采用了滑动窗口分块的策略对医学文档进行切片，每个 chunk 控制在 512 tokens，设置 128 tokens 重叠区域。相比于传统的固定分块方式，这种方式能有效保留上下文连续性，减少语义断裂。经过实验，在 BGE 向量召回模块中，我们的 Recall@5 提升了约 9.6%，对问答模块的 Top-3 正确率也提升了 6%。在 LangChain 中我们用 RecursiveCharacterTextSplitter 实现了这个策略，并做了自动化切片 pipeline。”

1.3 按自然段分块

一、什么是“按自然段分块”？

自然段分块是指按照文档中的自然段落进行切分，一般以换行符（如 `\n\n`、`\r\n`）作为段落边界。

📌 示例：

text

复制编辑

段1: 这是药品的适应症介绍.....
段2: 用法用量建议每天三次.....
段3: 不良反应包括恶心、呕吐.....

每段语义相对独立，因此按自然段切分有利于保持上下文一致性。

二、优缺点分析

优点	缺点	
✔ 保持语义完整，语法句子不易被截断	✖ 某些自然段可能太短或太长，影响向量化统一性	
✔ 与医学/法规类结构性文档高度适配	✖ 非结构化文本（如用户评论）难以自然切分	
✔ 分块粒度更贴合文档真实语义	✖ 长自然段仍需二次分块（如分句、滑窗）	

三、完整代码实现（Python 版本）

✔ Step1：按自然段分块 + 控制最大长度

```
def split_by_paragraph(text, max_chunk_len=500):  
    """  
    将文本按自然段（换行）分段，合并小段，确保 chunk 不超过最大长度  
    """  
    raw_paragraphs = [p.strip() for p in text.split("\n\n") if p.strip()]  
    print(raw_paragraphs)  
    chunks = []  
    current_chunk = ""  
  
    for para in raw_paragraphs:  
        print(para)  
        print('-----')  
        # 合并段落，控制长度 长度不能超过最大长度  
        if len(current_chunk) + len(para) < max_chunk_len:  
            current_chunk += para + "\n\n"  
        else:  
            if current_chunk:  
                chunks.append(current_chunk.strip())  
            current_chunk = para + "\n\n"  
  
    if current_chunk:  
        chunks.append(current_chunk.strip())  
    return chunks
```

```

# 示例使用
text = """
    【适应症】
    本品适用于上呼吸道感染、支气管炎等。

    【用法用量】
    成人一次500mg，一日3次，儿童酌减。

    【注意事项】
    肝肾功能不全者慎用。

    【不良反应】
    可见恶心、呕吐、腹泻等。
    """ * 3

chunks = split_by_paragraph(text, max_chunk_len=50)

for i, chunk in enumerate(chunks):
    print(f"\n--- Chunk {i+1} ---\n{chunk}")
    print('-----')

```

✅ 四、在 LangChain 中使用自定义 Paragraph 分块器

LangChain 的 `TextSplitter` 支持自定义段落切割。可以继承 `TextSplitter` 类实现。

✅ 自定义段落分块类

```

from langchain.text_splitter import TextSplitter
from langchain.schema.document import Document

class ParagraphSplitter(TextSplitter):
    def __init__(self, max_chunk_size: int = 500, add_start_index=False):
        self.max_chunk_size = max_chunk_size
        self._add_start_index = add_start_index

    def split_text(self, text: str):
        raw_paragraphs = [p.strip() for p in text.split("\n\n") if p.strip()]
        chunks = []
        current = ""

        for para in raw_paragraphs:
            if len(current) + len(para) < self.max_chunk_size:
                current += para + "\n\n"
            else:
                if current:
                    chunks.append(current.strip())
                current = para + "\n\n"
        if current:
            chunks.append(current.strip())
        return chunks

```

```
# 示例使用
text = """
【适应症】
本品适用于上呼吸道感染、支气管炎等。

【用法用量】
成人一次500mg，一日3次，儿童酌减。

【注意事项】
肝肾功能不全者慎用。

【不良反应】
可见恶心、呕吐、腹泻等。
""" * 3

splitter = ParagraphSplitter(max_chunk_size=80)
docs = splitter.create_documents([text])

for i, doc in enumerate(docs[:3]):
    print(f"\n--- Chunk {i+1} ---\n{doc.page_content[:150]}...\n")
```

五、项目实践中的应用建议

场景	建议	
药品说明书、诊疗指南	强烈推荐使用 paragraph 分块（语义边界清晰）	
非结构文本（网页/评论）	可先用段落切，再二次处理	
自然段过长时	可以结合“段内滑动窗口分块”混合使用	

六、与其他策略对比（加分项）

项目	固定长度分块	滑动窗口分块	自然段分块
实现复杂度	★	★★	★★
语义完整性	★	★★	★★★
通用性	★★★	★★★	★★
推荐场景	所有	所有	结构清晰文本（如说明书）

七、面试题推荐话术

“在我们的医疗 RAG 系统中，文档如药品说明书具有明确结构，例如“适应症”、“不良反应”等模块。我采用了自然段分块策略，即按照两个换行符将文档切分为语义完整的段落，再控制最大字符长度不超过512字符。相比固定长度或滑动窗口方式，自然段切分更能保留医学表达的语义边界，使得向量检索更精准，最终在我们的人工评估中 Top-3 命中率提升约 13%。”

1.4 标题小结分块

一、什么是标题小结分块？

标题小结分块（又叫结构感知切分）是指：

按照文档的结构化信息（如标题、节标题、小结、编号段）进行分块，每个“标题段”及其对应正文组成一个语义完整的Chunk。

举例（以医疗说明书为例）：

text

复制

编辑

【适应症】

本品适用于上呼吸道感染、肺炎等细菌感染。

【用法用量】

成人一次500mg，一日3次；儿童酌减。

【不良反应】

可见恶心、腹泻、皮疹等。

→ 每个标题 + 正文 = 一个语义段 (chunk)

二、适用场景

场景	说明
医疗说明书、指南	【适应症】、【用法用量】等结构清晰
法律法规	第x条、第x款、第x项 开头的段落
教材、规范性文件	按“章节标题”、“小节标题”划分
markdown 文档	按 # 标题层级解析

三、为什么使用标题小结分块？（对比其他策略）

指标	固定长度	滑动窗口	自然段	标题小结分块
语义完整性	✗	✓	✓	✓✓✓
检索精度	中	高	高	极高（精准召回）
分块粒度	固定	可调	弹性	按语义块
实现复杂度	简单	简单	中等	稍复杂（需结构识别）
工程推荐度	★★	★★★★	★★★★	★★★★★（强烈推荐结构化文档使用）

✓ 四、详细策略

✦ 1. 标题识别

通过正则表达式、规则识别标题格式：

text

复制 编辑

- ① 【标题】样式：如 【适应症】、【不良反应】
- ② Markdown 样式：如 ## 用法用量、### 注意事项
- ③ 编号样式：1.2、一、二、第一章

✦ 2. 标题段组合

识别标题 → 将其后紧跟的正文合并成一段

- 直到下一个标题出现为止

✓ 五、代码实现（Python 正则+结构解析）

✓ 示例代码：提取“标题+段落”的结构

```
import re

def split_by_heading(text):
    pattern = re.compile(r"(\[[^\]]+\])") # 标题格式如【适应症】
    parts = pattern.split(text)

    chunks = []
    for i in range(1, len(parts), 2): # 从第一个标题开始，每两个为一组
        title = parts[i].strip()
        content = parts[i+1].strip() if i + 1 < len(parts) else ""
        chunk = title + "\n" + content
        chunks.append(chunk)
    return chunks

# 示例文本
text = """
【适应症】
用于上呼吸道感染、支气管炎等疾病。

【用法用量】
成人一次0.5g，一日3次。

【不良反应】
可能引起腹泻、恶心。
"""

chunks = split_by_heading(text)
for i, chunk in enumerate(chunks):
    print(f"\n--- chunk {i+1} ---\n{chunk}")
```

输出结构：

diff

复制 编辑

--- Chunk 1 ---

【适应症】

用于上呼吸道感染、支气管炎等疾病。

--- Chunk 2 ---

【用法用量】

成人一次0.5g，一日3次。

...

六、在 RAG 系统中的好处

模块	影响	
向量化	chunk 表达更聚焦，提高 embedding 准确性	
检索召回	可以用 query 精确命中“适应症”段落	
生成回答	保证 chunk 语义清晰，减少模型误解	
多段融合	可按标题聚合，比如拼接【适应症】和【用法用量】段落回答	

七、实际项目配置建议

参数	建议值	
标题正则	<code>`r"([^[ ]+)"`r"([^[]+)"</code>	
最大段落长度控制	每段控制在 512 tokens 内，如过长再切	
切分后索引标记	为每个 chunk 添加 <code>meta={"section": "适应症"}</code> 可增强后期 rerank/排序	
多文档支持	将每篇文档编号，以避免段落合并错误	

八、项目经验中的表达（面试推荐）

“我们处理的文档是结构化较强的药品说明书，其包含‘适应症’、‘用法用量’、‘不良反应’等小节，因此我采用了标题感知分块策略，利用正则表达式提取各标题段及其正文内容作为单个 chunk。相比固定长度切分，这种方法大大增强了语义聚焦能力和召回精度，最终在人工评估中，Top-3 召回准确率提高了 17%，并使生成模块更容易抓住关键医学要素进行回答。”

1.5 动态切块（上面方法的结合）

一、什么是动态分块（Dynamic Chunking）

动态分块是一种结合多个因素（如 token 数、句子/段落边界、标题结构等）进行灵活分块的策略。

它与固定长度或自然段不同，强调 **“不在语义上硬截断”**，确保：

- 每个 chunk 都在设定的最大 token 限制内
- 并且尽可能以**句子或语义为单位划分**

✓ 二、动态分块的目标

目标	说明
控制每个 chunk 的 token 长度	避免 LLM 超长截断或 embedding 溢出
保证语义完整性	避免切半句、半段、语义残缺
灵活兼容不同结构文档	自动处理长短段落、标题组合等

✓ 三、为什么要用动态分块？

项目痛点	动态分块怎么解决	📄
文本结构不统一（有的长、有的短）	自适应切分点，控制 chunk 长度	
chunk 太长超 embedding 上限	按 token 动态切，避免截断	
chunk 太短语义不完整	合并相邻段/句保证可读性	
无法确定如何切断句子	利用分句器 + token 控制动态切	

✓ 四、典型策略逻辑（结合 NLP 分句 + Token 计算）

text

📄 复制 ✎ 编辑

输入文本 → 按段落/句子分割 → 累加 token 数直到最大限制 → 组成一个 chunk → 滑动窗口

✓ 五、完整代码实现（使用 transformers 分词器 + 中文分句器）

你可以复制、运行以下代码：

✓ Step 1: 安装必要库

bash

📄 复制 ✎ 编辑

```
pip install transformers jieba
```

✓ Step 2: 动态分块函数实现

```
# 导入结巴分词库
import jieba

# 导入transformers库中的AutoTokenizer
from transformers import AutoTokenizer

# 定义分句函数，将文本按句号、感叹号、问号分句
def sentence_split(text: str):
    """中文简单分句（可替换更强句法工具）"""
    import re
    # 使用正则表达式分句
    sentences = re.split(r'(?<=[。！？])', text)
    print(sentences)
    # 去除空句子并去除句子首尾的空白字符
```

```
return [s.strip() for s in sentences if s.strip()]
```

定义动态切分函数，将文本切分成多个块，每个块的长度不超过`max_tokens`，块之间有`overlap_tokens`的重叠

```
def dynamic_chunk(text, tokenizer, max_tokens=512, overlap_tokens=128):
```

```
    # 将文本分句
```

```
    sentences = sentence_split(text)
```

```
    # 初始化块列表
```

```
    chunks = []
```

```
    # 初始化当前块
```

```
    current_chunk = []
```

```
    # 初始化当前块的token数量
```

```
    current_tokens = 0
```

```
    for sent in sentences:
```

```
        # 将句子编码为token列表
```

```
        tokens = tokenizer.encode(sent, add_special_tokens=False)
```

```
        # 获取当前句子的token数量
```

```
        token_len = len(tokens)
```

```
        # 如果当前 chunk + 新句子超限 → 切分
```

```
        if current_tokens + token_len > max_tokens:
```

```
            # 将当前块编码为字符串并添加到块列表中
```

```
            chunks.append(tokenizer.decode(
                tokenizer.encode("".join(current_chunk),
```

```
add_special_tokens=False)
```

```
            ))
```

```
            # 启动重叠策略（从末尾截 overlap_tokens）
```

```
            overlap = []
```

```
            while current_tokens > overlap_tokens and current_chunk:
```

```
                # 从当前块中弹出一个句子
```

```
                overlap_sent = current_chunk.pop()
```

```
                # 减少当前块的token数量
```

```
                current_tokens -= len(tokenizer.encode(overlap_sent,
add_special_tokens=False))
```

```
                # 将弹出的句子添加到重叠列表中
```

```
                overlap.insert(0, overlap_sent)
```

```
            # 重置当前块为重叠列表的副本
```

```
            current_chunk = overlap.copy()
```

```
            # 重新计算当前块的token数量
```

```
            current_tokens = sum(len(tokenizer.encode(s,
add_special_tokens=False)) for s in current_chunk)
```

```
        # 将当前句子添加到当前块中
```

```
        current_chunk.append(sent)
```

```
        # 增加当前块的token数量
```

```
        current_tokens += token_len
```

```
    # 如果当前块不为空，将其添加到块列表中
```

```
    if current_chunk:
```

```
        chunks.append("".join(current_chunk))
```

```
    # 返回切分后的块列表
```

```
    return chunks
```

```
if __name__ == '__main__':
```

```
# 加载预训练的分词器
tokenizer = AutoTokenizer.from_pretrained("bert-base-chinese") # 你也可以用
bge-base-zh、Qwen等

# 模拟长文
text = """
阿莫西林适用于敏感细菌引起的下列感染：如呼吸道感染、泌尿系统感染、皮肤和软组织感染等。使用本品前应明确细菌敏感性。
常见不良反应有胃肠道不适、皮疹等。严重者应及时停药。
成人一次0.5克，一日3次；儿童按体重酌减。
本品应避光、密封保存。
""" * 5 # 模拟长文

# 调用动态切分函数，将文本切分成多个块
chunks = dynamic_chunk(text, tokenizer, max_tokens=100, overlap_tokens=30)

# 打印每个块及其token数量
for i, chunk in enumerate(chunks):
    print(f"\n--- Chunk {i+1} ({len(tokenizer.encode(chunk))} tokens) ---\n{chunk}")
```

六、工程实战中如何配置动态分块？

参数	推荐值	说明
max_tokens	512 或 384	chunk 最大 token 长度
overlap_tokens	64~128	滑动窗口重叠，保证上下文连续
分句方式	中文：正则 or LTP，英文：nltk or spaCy	分句需语义准确

七、在 RAG 系统中的优势

模块	动态分块的好处	📄
向量化 (Embedding)	每个 chunk token 长度受控，embedding 性能稳定	
检索召回	语义切分更精准，召回更 relevant	
LLM 生成	避免语义断裂 + 减少无用信息干扰	
多文档融合	chunk 粒度一致，更易排序/融合	

八、面试推荐答法（可直接背诵）

“在我们的医疗问答项目中，原始文档结构复杂，存在长短句混杂、段落不规范等问题。为了保证 chunk 语义完整性且 token 不超限，我采用了动态分块策略。具体方法是先对文本分句，然后使用 tokenizer 控制每个 chunk 的 token 数量不超过 512，设置 128 token 重叠窗口。这样既能保持语义连续性，又能与下游向量化和 LLM 推理兼容，最终使检索召回率📈提升了 13%，生成问答准确度提升了约 9%。”

✅ 三、分块策略对检索/生成的影响

模块	分块粒度的影响
向量检索	粒度太大 → embedding 不准; 太小 → 语义碎片无用
文档召回质量	粒度合理 → 更准确召回 (覆盖性、精度都提升)
LLM 生成答案	粒度不合理 → 内容不连贯、前后文缺失
多文档融合效率	Chunk 过多 → 太多无关 chunk 干扰 → 降低准确性

所以选择合适的 chunking 策略非常关键!

✅ 四、实际应用中的推荐配置

最常见配置:

yaml

```
chunk_size: 512 # 每块 512 tokens
chunk_overlap: 128 # 每块重叠 128 tokens
split_method: 'sentence+token' # 按句子语义 + token 数做切分
```

复制 编辑

结合 LangChain 示例:

python

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=128,
    separators=["\n\n", "\n", ".", "。", "!", "?", " "]
)
docs = text_splitter.split_documents(raw_docs)
```

复制 编辑

✅ 五、面试中可用的标准答法模板

“在我们 RAG 医疗问答系统中，原始文档如药品说明书、疾病百科通常篇幅较长，为了提升向量召回的准确率，我们使用了分块策略：
我采用了基于 token 数 + 语义断句 的复合分块方式，每块控制在 512 tokens，且设置了 128 tokens 重叠，避免知识片段断裂带来召回错失。
经过评估，这种方式相比于固定块长分割，提升了 Top-5 检索命中率约 11%，尤其在实体较长和语义衔接要求高的医疗场景下效果更好。”

六、补充：Chunk 与知识融合

- 在 RAG 知识融合阶段（Top-K 多文档合并），如果 chunk 语义断裂严重，会导致生成质量下降；
- 可通过“重排序+去重+跨段拼接”等手段优化 chunk 合并顺序与内容；

七、推荐你说的亮点/加分点

加分项	展示能力
动态按标题语义分块	“我写了一个针对说明书的结构化 parser，按‘适应症/用法’等分块，效果显著”
分块评估实验	“我评估了 256/512/1024 分块对召回命中率的影响，发现512是最优”
对比不同分块策略	“我们比较了 Fixed vs Overlap vs SentenceSplit，在 QA 保真度上 SentenceSplit 更佳”

1.6 最常见的分块策略

固定长度分块（Fixed-Length Chunking）

一句话总结：

将原始文本按固定 token 数（如 256、384、512）切分，不考虑语义、句子、段落等边界。

为什么它最常见？

优势	说明
实现简单	不需要分句器、正则、结构解析器
工程通用	适配任何结构的文本（网页、说明书、通话记录）
LLM 友好	控制 token 数方便 embedding、推理窗口管理
HuggingFace、LangChain 默认都支持	如 RecursiveCharacterTextSplitter 默认即采用近似策略

固定长度分块策略怎么做？

- 将文本按字符或 token 切成定长的块
- 每块长度为 max_tokens（如 512），并可以设置 overlap_tokens（如 50）防止语义断裂

✓ 示例 (Token长度为单位)：

text

复制 编辑

原始文本：
[abcdefghijklmnopqrstuvwxyz1234567890abcdefghijklmnopqrstuvwxyz...]

→ 固定长度为 20，重叠长度为 5

Chunk 1: tokens[0:20]
Chunk 2: tokens[15:35]
Chunk 3: tokens[30:50]
...

✓ 代码示例 (以 tokenizer 控制 token 切分)：

python

复制 编辑

```
def fixed_token_chunk(text, tokenizer, max_tokens=512, overlap_tokens=50):
    tokens = tokenizer.encode(text, add_special_tokens=False)
    chunks = []

    start = 0
    while start < len(tokens):
        end = start + max_tokens
        chunk_tokens = tokens[start:end]
        chunk_text = tokenizer.decode(chunk_tokens)
        chunks.append(chunk_text)
        start += max_tokens - overlap_tokens # 滑动窗口
    return chunks
```

✓ 使用场景

类型	是否适合	原因	📄
医疗问答	✓	医学句子多，长度较规整，便于分割	
法规条文	✓	条款长度平均	
网页文本	✓	页面结构不规则，用固定更稳	
聊天记录	✓	无结构语料，无法用标题划分	
Markdown / 说明书	✗	建议用标题感知切分更优	
超长段落	✗	建议用滑窗或动态切分防止截断	

✅ 和其他策略对比

分块策略	是否常用	特点	📄
✅ 固定长度	✅ 最常用	简单、快速、普适	
滑动窗口	✅	适合无语义边界文本	
动态分块	❖	智能但复杂	
按自然段	❖	适合结构性强的文档	
标题感知切分	❌ (少数项目使用)	精准、语义完整，但需要结构规则清晰	

✅ 面试推荐说法（你可以背）：

“在我们的 RAG 系统中，最常用的是**固定长度分块**策略。我们会根据模型支持的最大 token 数（如 512）将文本切分成定长片段，同时通过设置滑动窗口（如 128 token 重叠）来保证 chunk 之间的语义连续性。这种方法在医疗、法律等文档结构不稳定或无语义断点的场景中效果稳定，部署简单，是目前工程落地中最常见的切分方式之一。”

✅ 总结一句话：

固定长度分块是“最简单、最常见、最通用”的 RAG 分块策略，适合所有文本的首选 baseline。